

Object-Oriented Programming

Constructor functions, prototypes, ES6 classes, inheritance with extends/super, and prototypal inheritance.

Constructor Functions

Prototypes

Classes

Inheritance

1 Object-Oriented Programming (OOP)

OOP is a way of organizing code around **objects** that contain:

Properties → data

```
name: "Natraj"  
color: "black"  
price: 10
```

Methods → behavior

```
write: function () { ... }
```

```
let pencil = {  
  name: "Natraj",  
  color: "black",  
  price: 10,  
  
  write: function () {  
    // ...  
  }  
};
```

2 Constructor Function

Before ES6 classes, constructor functions were commonly used to create multiple similar objects.

```
function CreatePencil(name, color, price) {  
  this.name = name;  
  this.price = price;  
  this.color = color;  
}
```

this represents the **new object** being created.

Objects are created using **new** :

```
let pencil1 = new CreatePencil("Natraj", "black", 10);  
let pencil2 = new CreatePencil("Doms", "red", 10);
```

CreatePencil()



pencil1

Natraj · black · 10



pencil2

Doms · red · 10

3 Methods in Constructor Functions

You can put methods directly inside the constructor:

```
function CreatePencil(name, color, price) {  
  this.name = name;  
  this.price = price;  
  this.color = color;  
  
  this.write = function (text) {  
    let h1 = document.createElement("h1");  
    h1.textContent = text;  
    h1.style.color = color;  
    document.body.append(h1);  
  };  
}
```

Problem

Every new object gets its **own copy of the function**.

pencil1
→ own write()

pencil2
→ own write()

pencil3
→ own write()

This can unnecessarily consume memory.

4 Prototypes

A **prototype** is an object from which other objects can inherit properties and methods.

Instead of putting shared methods directly inside the constructor, put them on the constructor's **prototype**.

```
function CreatePencil(name, color, price) {  
  this.name = name;  
  this.price = price;  
  this.color = color;  
}  
  
CreatePencil.prototype.write = function (text) {  
  let h1 = document.createElement("h1");  
  
  h1.textContent = text;  
  h1.style.color = this.color;  
  
  document.body.append(h1);  
};
```

Now create objects:

```
let pencil1 = new CreatePencil("Natraj", "black", 10);
let pencil2 = new CreatePencil("Doms", "red", 10);

pencil1.write("Hello");
pencil2.write("JavaScript");
```

CreatePencil.prototype.write()

↑ shares

pencil1

↑ shares

pencil2

They share the same prototype method instead of each storing a separate copy.

5 Prototype Properties

Properties can also be added to the prototype.

```
CreatePencil.prototype.company = "Doms";
```

Now:

```
console.log(pencil1.company);
console.log(pencil2.company);
```

Both can access: Doms

The property isn't directly stored on each pencil object. JavaScript looks through the object's **prototype chain** when it can't find the property directly.

6 class

ES6 introduced the **class** syntax, which provides a cleaner way to create objects and inheritance.

```
class CreatePencil {
  constructor(name, color, company) {
    this.name = name;
    this.color = color;
    this.company = company;
  }

  write(text) {
    let h1 = document.createElement("h1");

    h1.textContent = text;
    h1.style.color = this.color;

    document.body.appendChild(h1);
  }

  erase() {
    document.body.querySelectorAll("h1").forEach((element) => {
      if (element.style.color === this.color) {
        element.remove();
      }
    });
  }
}
```

pencil.js

```
}  
}
```

Create objects:

```
let p1 = new CreatePencil("rx1", "black", "doms");  
let p2 = new CreatePencil("rx2", "red", "nataraj");
```

7 constructor()

Inside a class:

```
constructor(name, color, company) {  
  this.name = name;  
  this.color = color;  
  this.company = company;  
}
```

The constructor runs automatically when `new` is used.

```
let p1 = new CreatePencil("rx1", "black", "doms");
```

`new CreatePencil(...)`



constructor runs



new object is created



properties are initialized

8 Methods in Classes

Methods are written directly inside the class:

```
write(text) {  
  // code  
}  
  
erase() {  
  // code  
}
```

You don't write `function write() {}` inside the class.

NOTE

Methods defined in a class are placed on the class's prototype, rather than copied as a separate function onto every instance.

9 Inheritance

Inheritance allows one class to receive properties and methods from another class.

```
class User {
  constructor(name, address, username, email) {
    this.name = name;
    this.address = address;
    this.username = username;
    this.email = email;
    this.role = "user";
  }

  checkRole() {
    return `you are a ${this.role}`;
  }

  write(text) {
    let h1 = document.createElement("h1");

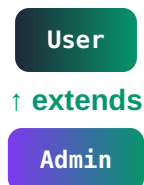
    h1.textContent = `${this.name}: ${text}`;

    document.body.appendChild(h1);
  }
}
```

Admin can inherit from **User**:

```
class Admin extends User {
  // ...
}
```

extends establishes inheritance.



10 super()

When a child class has its own constructor, it must call the parent constructor using **super()** before using **this**.

```
class Admin extends User {
  constructor(name, address, username, email) {
    super(name, address, username, email);

    this.role = "admin";
  }
}
```

super() calls the **parent class constructor**. Therefore the inherited properties are initialized:

name · address · username · email

Then Admin changes:

```
this.role = "admin";
```

11 Adding Child-Specific Methods

The child class can have methods that the parent doesn't have.

```
class Admin extends User {  
  
  remove() {  
    document.querySelectorAll("h1").forEach(function (element) {  
      element.remove();  
    });  
  }  
}
```

So an Admin has:

↑ Inherited from User

checkRole()
write()

✨ Own method

remove()

12 Creating User and Admin Objects

```
let u1 = new User(  
  "Harsh",  
  "Bhopal",  
  "async123",  
  "a@aa.com"  
);  
  
let u2 = new User(  
  "Abrar",  
  "Srinagar",  
  "abrar123",  
  "abrar@gmail.com"  
);  
  
let a1 = new Admin(  
  "Ali",  
  "Mumbai",  
  "ali_admin",  
  "ali@gmail.com"  
);
```

a1 can use both:

```
a1.write("Hello");  
a1.checkRole();  
a1.remove();
```

because **Admin** inherits from **User**.

13 Prototypal Inheritance

JavaScript's inheritance is fundamentally **prototype-based**. Objects can directly inherit from other objects.

```
let coffee = {
  color: "dark",

  drink: function () {
    console.log("drinking .....");
  }
};
```

Create another object whose prototype is `coffee` :

```
let arabiataCoffee = Object.create(coffee);
```

Now:

```
arabiataCoffee.drink();
```

works even though `drink` was not directly defined inside `arabiataCoffee` .



14 Object.create()

```
let child = Object.create(parent);
```

Creates a new object whose prototype is `parent` . Example:

```
let coffee = {
  color: "dark",

  drink: function () {
    console.log("drinking .....");
  }
};

let arabiataCoffee = Object.create(coffee);
```

Now `arabiataCoffee.drink();` can access `drink` through `coffee` .

OOP QUICK REVISION

Constructor Function

```
function User(name) {
  this.name = name;
}

let u1 = new User("Abrar");
```

Prototype

```
User.prototype.sayHi = function () {  
  console.log("Hi");  
};
```

Shared by instances.

Class

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
  
  sayHi() {  
    console.log("Hi");  
  }  
}
```

Inheritance

```
class Admin extends User {  
}
```

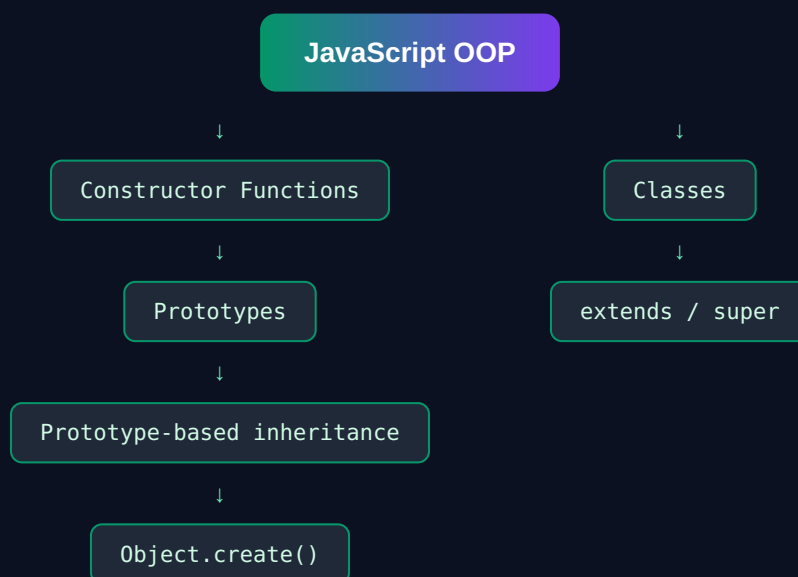
Parent Constructor

```
super(...);
```

Object → Object Inheritance

```
let child = Object.create(parent);
```

The Big Picture



★ IMPORTANT

JavaScript is fundamentally prototype-based. class is cleaner syntax built on top of JavaScript's prototype system.

✨ Constructors build the object, prototypes make the sharing cheap, and class is just a nicer face on both.